

(Ethics: Any behavior on any homework or exam that could be considered copying or cheating will result in an immediate zero on the assignment for all parties involved and will be reported to academichonesty@iit.edu. See the IIT Code of Academic Honesty, <https://web.iit.edu/student-affairs/handbook/fine-print/code-academic-honesty>

1. Let $W \equiv \{\text{inv } p\}\{\text{bd } t\} \text{ while } B \text{ do } S \text{ od}$ and $\vdash_{tot} \{\text{inv } p\}\{\text{bd } t\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}$. For each of the following statements about bound expression, decide true or false and justify your answer briefly.
 - a. Let $\sigma \models p$, then $\perp_d \notin M(W, \sigma)$.
 - b. The value of t can be negative after the execution the last iteration of W .
 - c. $sp(p \wedge B \wedge t = t_0, S) \Rightarrow t < t_0$
 - d. $p \wedge t > 0 \Rightarrow B$
2. Let $W \equiv \{\text{inv } p\}\{\text{bd } t\} \text{ while } k \leq n \text{ do } x := x - 2; k := k + 1 \text{ od}$, and we know that $p \equiv n \geq 0 \wedge 0 < C \leq k \leq n + C$, where C is a constant. For each of the following expressions, decide whether it can be the bound expression for W . Justify your answers.
 - a. $x - k + n$
 - b. $n - k$
 - c. $n - k + C$
 - d. $k - C$
 - e. $2^n \cdot 2^{(C - k)}$
3. Here is a minimal proof outline that we see in Assignment 5.

```

{y ≥ 1} x := 0; r := 1;
{inv 1 ≤ r = 2^x ≤ y}
while 2 * r ≤ y do r := 2 * r; x := x + 1 od
{r = 2^x ≤ y ≤ 2^(x + 1)}

```

Now, find a reasonable bound expression for the while loop in this minimal proof outline, and create a full proof outline **under total correctness** for it. Please use the **Backward Assignment Axiom** to prove all the assignment statements. Give a brief explanation on each logical implication used in the proof outline.

4. Consider predicate $q \equiv y \geq 0 \wedge x = 2 * y \leq n < 3 * (y + 1)$, where n is a named constant and x and y are variables, as the postcondition of a loop. Using the technique **replacing a constant in q by a variable**, create 5 possible candidates for the loop invariant p and their corresponding loop condition B .

Hint: You won't be able to figure out the range of the introduced variable only from the postcondition, so you don't need to include that in the loop invariant candidates.

5. Consider predicate $q \equiv (y \geq 0) \wedge (z = 2^y) \wedge (2^y \leq x) \wedge (x < 2^{(y + 1)})$ as the postcondition of a loop. Using the technique **dropping off one conjunct in q** , create 4 possible candidates for the loop invariant p and their corresponding loop condition B .

6. We will create a program that calculates the factorial of natural number n and stores the result as $x = fac(n)$. Here, function $fac(n)$ is the function that gives the factorial of n :

$$fac(n) = \begin{cases} 1, & \text{if } n = 0 \\ n * fac(n - 1), & \text{if } n > 0 \end{cases}$$

Answer the following questions. Justify your answer briefly.

- What are the pre- and post- conditions needed for this program?
- Create a loop invariant p by replacing the named constant n by a variable y in the postcondition (what can be the range of variable y ?) and create the corresponding loop condition B .
- Find a bound expression t for this program.
- Create a full proof outline under the total correctness for this program. Don't forget to initialize variables x and y before the loop starts, find reasonable values for them. You don't have to justify logical implications you used in the proof outline.

(Note that, questions 7 and 8 are about substitutions in arrays. This topic is not covered in Midterm 2; it will be covered in the final exam.)

7. Find an optimized precondition p and create a full proof outline for the following minimal proof outline. You may assume that $0 \leq i < size(b)$, $0 \leq j < size(b)$, $0 \leq k < size(b)$ so you don't need to worry about the array-index-out-of-bounds error.

$$\{p\} b[i] := b[j]; b[j] := b[k] \{b[i] > b[k]\}$$

8. Create a full proof outline for the following minimal proof outline. You may assume that $0 \leq j < size(b)$, $0 \leq k < size(b)$, so you don't need to worry about the array-index-out-of-bounds error.

$$\{k < b[k] < b[j]\} b[b[k]] := b[j] \{b[k] \neq b[j]\}$$